

Software Quality - W9.A7

Test Oracles

Riva Daniele Mat.: 902275

May 14, 2026

1 Method 1: isValidIndex(Object[] array, int index)

```
1  /**
2   * Checks if the specified index is within the bounds of the array.
3   *
4   * @param array the array to check, not null
5   * @param index the index to check
6   * @return true if index is valid (0 <= index < array.length)
7   * @throws IllegalArgumentException if array is null
8   */
9  public static boolean isValidIndex(Object[] array, int index) {
10     if (array == null) {
11         throw new IllegalArgumentException("Array cannot be null");
12     }
13     return index >= 0 && index < array.length;
14 }
```

Preconditions

- array != null;

Postconditions

- true ? result == (index >= 0 && index < array.length) : true;

Exceptional conditions

- array == null;

2 Method 2: parsePositiveInt(String str)

```
1  /**
2   * Parses a positive integer from the string.
3   * Leading/trailing whitespace ignored.
4   *
5   * @param str the string to parse, not null
6   * @return the integer value
7   * @throws NumberFormatException if not a valid positive integer
8   */
9  public static int parsePositiveInt(String str) {
10     String trimmed = str.trim();
11     if (trimmed.isEmpty() || !trimmed.matches("\\d+")) {
12         throw new NumberFormatException("Invalid positive integer");
13     }
14     int value = Integer.parseInt(trimmed);
15     if (value <= 0) {
16         throw new NumberFormatException("Value must be positive");
17     }
18     return value;
19 }
```

19 }

Preconditions

- `str != null;`

Postconditions

- `true ? result > 0 : true;`

Exceptional conditions

- `str.trim().isEmpty() || !str.trim().matches("\\d+") || Integer.parseInt(str.trim()) <= 0;`

3 Method 3: filterEvens(List<Integer> list, int capacity)

```
1  /**
2   * Filters list to even numbers only.
3   *
4   * @param list input list of integers, not null
5   * @param capacity max size of output list
6   * @return filtered list, never null
7   * @throws IllegalArgumentException if list null or capacity < 0
8   */
9  public static List<Integer> filterEvens(List<Integer> list, int capacity) {
10     if (list == null || capacity < 0) {
11         throw new IllegalArgumentException("Invalid input");
12     }
13     List<Integer> result = new ArrayList<>();
14     for (Integer num : list) {
15         if (num != null && num % 2 == 0) {
16             if (result.size() < capacity) {
17                 result.add(num);
18             } else {
19                 break;
20             }
21         }
22     }
23     return result;
24 }
```

Preconditions

- `list != null && capacity >= 0;`

Postconditions

- `true ? result != null && result.size() <= capacity : true;`
- `true ? result.stream().allMatch(n -> n != null && n % 2 == 0) : true;`

Exceptional conditions

- `list == null || capacity < 0;`

4 Method 4: isValidUsername(String username)

```
1  /**
2   * Validates a username for account creation.
3   *
4   * A valid username:
5   * - is not null
6   * - has length between 4 and 12 characters inclusive
```

```

7  * - starts with a letter
8  * - contains only letters, digits, or underscore
9  *
10 * @param username the username to validate
11 * @return true if the username is valid, false otherwise
12 * @throws IllegalArgumentException if username is null
13 */
14 public static boolean isValidUsername(String username) {
15     if (username == null) {
16         throw new IllegalArgumentException("Username cannot be null");
17     }
18     if (username.length() < 4 || username.length() > 12) {
19         return false;
20     }
21     char first = username.charAt(0);
22     if (!Character.isLetter(first)) {
23         return false;
24     }
25     for (int i = 0; i < username.length(); i++) {
26         char c = username.charAt(i);
27         if (!(Character.isLetterOrDigit(c) || c == '_')) {
28             return false;
29         }
30     }
31     return true;
32 }

```

Preconditions

- username != null;

Postconditions

- true ? result == (username.length() >= 4 && username.length() <= 12 && Character.isLetter(username.charAt(0)) && username.chars().allMatch(c -> Character.isLetterOrDigit(c) || c == '_')) : true;

Exceptional conditions

- username == null;